

StateSync – Recording Notes

Five minutes. ~700 spoken words at a natural pace. That's your entire budget — write to it, don't discover it on take three.

Your demo executes in 3.4 seconds total, so **nothing on screen will ever pace you**. That's freedom and it's a trap: with no loading to wait through, the instinct is to rush. Every pause has to be deliberate.

Part 1 – Before you press record

The machine

- ☐ **Do Not Disturb on.** Slack, mail, WhatsApp Web, calendar alerts — all of it. A notification banner mid-take means starting over.
- ☐ **Close everything you're not showing.** One terminal, one browser tab if you're showing the UI.
- ☐ **No background processes.** No `make verify` running, no dev server you forgot about. That's what produced the 929-second smoke reading.
- ☐ **Terminal font at 16–18pt.** Test it: shrink the recording to 50% on screen and check the verdict table is still readable. If it isn't, a judge on a laptop can't read it either.
- ☐ **Window at 1600×900 or 1920×1080.** Not fullscreen on a 27" monitor — the text ends up tiny in the export.
- ☐ **Clear the scrollbar** (`clear` or ⌘K) before every take.
- ☐ **Dark terminal theme**, matching the UI. Consistency reads as intentional.
- ☐ **Battery plugged in.** Thermal throttling mid-take is a real thing.

Audio matters more than video

Judges will forgive a mediocre screen capture. They will not sit through audio they have to strain at.

- Record in a small room with soft furnishings — bed, curtains, clothes. Not a bare-walled room, not a kitchen.
- **Turn off the fan and the AC.** Yes, in Hyderabad in September. Record in short takes and turn it back on between them.
- Wired earbuds with a mic beat a laptop mic. Phone earbuds are fine.
- Mic 6–8 inches from your mouth, slightly off to the side so plosives don't pop.
- **Record 10 seconds of silence first** and listen back. If you hear hum or hiss, fix it before doing a full take.

The tools

macOS: ⌘⇧5 → Record Selected Portion → select the terminal window with a little margin. Free, no setup, exports MP4.

OBS Studio if you want more control (free, all platforms). Display Capture source, 1080p, 30fps is plenty.

Don't use a webcam overlay. Nobody needs to see you. The screen is the content, and a floating head is one more thing to go wrong.

Part 2 – The script

Timings are targets, not laws. Total 5:00.

Read each opener **as written** – those exact sentences have been through several rounds. Everything else you can paraphrase in your own words.

BEAT 1 – The problem (0:00–0:45)

Show: `razorpay/razorpay-magento#208` on screen, or the README's problem section.

Say – all five links, in order. Do not skip one:

"Payment gateways sync with merchant systems using webhooks. Webhooks deliver at-least-once, with no ordering guarantee, and after 24 hours of failure they give up permanently.

So a merchant turns webhooks on and gets duplicate orders. Turns them off and loses paid orders. This is Razorpay's own open issue – there is no correct setting.

The result is that money moved and the merchant's database doesn't know. Today, a human reconciles that by hand.

I built the agent that does the first seventy percent of that."

Pacing: slow. This is the only beat where a listener is building context from nothing. Everything after depends on them having understood this.

BEAT 2 – Architecture, and the pre-empt (0:45–1:35)

Show: the architecture diagram, or `ARCHITECTURE.md`.

Say:

"Three sources hold a view of the same transaction — the gateway, the merchant's order store, and the ledger. StateSync reconciles all three, classifies every disagreement, repairs what's safe, and escalates the rest.

Four of the six divergence classes are exact set operations. Rules handle those. When I report a hundred percent detection on them, that's the expected floor, not an achievement — a set difference that failed to find a set difference would be a bug.

The interesting class is amount mismatch, where the numbers don't line up and several explanations produce nearly the same figure. That's where a model is used — and only there.

The model proposes explanations. A deterministic verifier decides whether they're accepted."

This is the beat that compresses if you're running long. Cut the first paragraph, keep the other two.

Why the pre-empt matters: you say "100% is the floor" before a reviewer thinks "100% is suspicious." Volunteered, it's rigour. Discovered, it's a hole.

BEAT 3 — Correct refusal (1:35–2:05)

Show: case 7 in the output, and `exceptions.csv` with its header comment.

Say:

"Two orders. Same customer, same amount, two seconds apart. It looks exactly like a duplicate. It isn't — two separate payments.

The system does not merge them. And it produces **no row** in the exception list, because a divergence the system correctly declined to act on is not an exception. The CSV says that in its own header.

Knowing when not to act is harder than acting."

Pause after the last line. One full second. It's the thesis of the project.

BEAT 4 — Every guard fired (2:05–3:00) ★ your strongest 55 seconds

Show: the verdict table. If you show the UI at all, show `pay_hc06`.

Say, as the `llm_calls: 0` renders:

"Every model response is cached and committed, so this demo is reproducible and needs no API key. The cold run happened once — eight calls, 1,493 milliseconds mean, on `openai/gpt-`

oss-120b — recorded in the manifest with the date and the model. What you're watching replays that."

Then the table:

"The model proposed sixteen explanations. The verifier accepted four and refused twelve — across four different checks. Arithmetic failed. Artifact missing. Range violation. Rate inconsistent.

All four fired on real model output. None of them is decorative."

If showing pay_hc06 on the UI, add:

"Look at this one. The arithmetic is perfect — sixty equals sixty, green tick. And it's still rejected, because the reserve it cites doesn't exist. The model produced maths that works and evidence that doesn't. That's what the guard is for."

Do not say: "the model was accurate", "we achieved X%", or anything implying a rate you can generalise from. Say counts. Sixteen proposed, twelve refused.

Do not mention the retry-resolved figure. It moved between cache builds at temperature 0.7. It's in the eval output with its sample size, which is the right place for it.

BEAT 5 — Idempotency (3:00–3:50)

Show: three runs, live.

Say — and pause between each result:

"Run the batch. A hundred and twenty-four repairs."

(pause)

"Run the identical batch again. **Zero writes.**"

(pause)

"Now flush Redis entirely — delete the cache the system uses to track what it's already done. Run it a third time. **Still zero.** The database unique constraint caught every duplicate."

(pause)

"Two independent layers, and I mutation-tested them: I broke each one separately to confirm the other catches it."

This renders in 1.6 seconds and carries the credibility of the entire submission. **Slow it down.** Three separate results, three separate pauses.

If your demo runs the in-memory store rather than Postgres, say so: *"the constraint is real and tested – this run uses the in-memory store."* One clause. Never narrate something the terminal isn't doing.

BEAT 6 – Fail closed (3:50–4:20)

Show: `eval/tamper.py`, then the run halting.

Say:

"The audit ledger is a SHA-256 hash chain. Tamper with one entry –"

(run it)

"– and the run halts before the first repair. Non-zero exit. **Zero writes.**

It verifies before repairing, not after, because verifying afterwards means the writes already happened.

And I should tell you: this check existed and worked for most of the build, and **nothing called it.** The chain verified correctly, the tamper detection was right, and the result was never consumed. Every component test passed. The failure lived in the seam between two correct pieces."

That last paragraph is worth more than the feature. It's a real bug, found late, told honestly.

CLOSE – What it can't do (4:20–5:00)

Show: the README limits section.

Say:

"What this doesn't do.

Settlement gap is in the taxonomy and ships as **not implemented** – with the reason, and no accuracy figure attached.

The verifier is sound but not complete. It catches fabricated explanations. It cannot catch a missing one – if the model never proposes a valid explanation, the system reports that nothing verified, and it can't tell you what it didn't think of.

The ambiguous sample is seven cases. That's enough to show the architecture works. It is not enough to characterise a rejection rate.

And the same bug appeared eleven times in this build — a value computed correctly in one place and then re-derived, misrouted, or ignored somewhere else. They're catalogued in the repo, with the count generated from the file, because writing 'eleven' by hand would have been the twelfth."

End there. No summary, no thank-you, no "hope you liked it." The last thing they hear is you listing your own limits with a joke that lands. That's the note to leave on.

Part 3 – Rehearsal protocol

Do not rehearse in your head. Out loud, at full volume, standing up. Silent rehearsal hides every place you stumble.

Pass 1 – timing only

Run `DEMO_PAUSE=0 ./scripts/demo.sh` and talk over it with a stopwatch. Note where you overrun. Beat 2 is the one that compresses.

Pass 2 – the numbers

Say every figure out loud. `1,493 milliseconds` and `openai/gpt-oss-120b` are the two that trip people. Read numbers off the screen where you can — a figure you read is never stale, a figure you memorised might be.

Pass 3 – record it

Even if you plan another take. Watching yourself is the fastest way to find that you say "um" before every number or that your terminal font is too small.

Then record take four and ship it. Not take eight. Diminishing returns arrive faster than you think, and tiredness in your voice is more damaging than a small stumble.

Part 4 – Recording workflow

1. Fresh terminal, `clear`, DND on, fan off
2. Start the recorder, **wait three seconds before speaking** — gives you clean handles to trim

3. Run through all six beats without stopping
4. **If you stumble: pause, breathe, repeat the sentence.** Don't restart the take. You can cut it in edit.
5. Three seconds of silence at the end before stopping
6. Watch it back at normal speed, once, with headphones

Editing: trim the head and tail, cut any stumble-repeats. Nothing else. No music, no titles, no zoom effects, no captions. This is an engineering demo – production polish reads as compensation.

Export: 1080p MP4. Check the file plays somewhere that isn't your machine before you submit it.

Part 5 – Questions you should be ready for

Not in the video. For the conversation after.

"What happens at n=70 ambiguous cases?"

"I don't know. Seven cases shows the architecture works and every guard fires. It isn't enough to characterise a rejection rate. Scaling the hard-case generator is the next thing I'd build."

"Why not let the model make the decision?"

"Every repair class has a dangerous failure mode – misclassify a refund and you refund twice. A decision that has to be auditable can't come from something that gives a different answer on the same input twice."

"Your data is synthetic."

"The track asked for synthetic. And it's a strength here: I inject the divergences myself, so ground truth is exact. No labelling, no arguing about the answer key."

"Why is there no way to add a transaction in the UI?"

"Transactions come from the gateway and the order store. A reconciler that lets you type entries creates a third source of truth. The write path I'd add next is acting on an exception – mark resolved with a note, and add the fee schedule entry that `pay_hc17` is literally asking for."

"Did the AI actually help?"

"Two of six ambiguous cases resolved. Seventy-five percent of proposals refused. The defensible claim is that the guard is load-bearing, not that the model is accurate."

"What's the biggest weakness?"

"The verifier is sound but not complete. It catches fabrication and can't catch omission. That's why hypotheses are pooled rather than replaced — the model can only add explanations, never remove doubt that the deterministic set already established."

Part 6 – Failure modes to avoid

Don't	Why
Rush because nothing is loading	3.4s of execution, 5 minutes of narration — all pacing is yours
Say "we"	You built it solo. Say "I"
Apologise for anything	State limits flatly. "Not implemented, with the reason" is confident. "Unfortunately I didn't get to..." isn't
Read the README aloud	They can read. Narrate what's on screen and why it matters
Add music or titles	Production polish on an engineering demo reads as compensation
Do more than four takes	Tiredness in your voice costs more than a small stumble
Mention the retry figure	$n=1$ at temperature 0.7. It's in the eval output where it belongs
Claim a rate from seven cases	Say counts: sixteen proposed, twelve refused

The one-sentence version

If you get 30 seconds instead of 5 minutes:

"Payment gateways and merchant databases keep separate records of the same transaction and they drift — Razorpay employs people to reconcile that by hand. I built the agent that does it: it finds the disagreements, fixes what's provably safe, and escalates the rest. Where the gap is genuinely ambiguous a model proposes explanations, and a separate verifier only accepts the ones that reconcile to the exact paisa. It refused twelve of sixteen proposals — and I can show you every one it got wrong and why."